

ИНТЕРФЕЙСИ – Задача Ветрове

```
Microsoft Visual Studio Debug Console

=== 5-ТЕ НАЙ-ИЗВЕСТНИ ВЯТЪРА ===
=====

=== ЗЕФИР ===
Посока: Западен вятър.
Скорост: 1-5 m/s (лек вятър).
Тип: Локален, приятен лек ветрец
Описание: Лек западен вятър, характерен за Средиземноморието.

=== БРИЗ ===
Посока: Денем от морето към сушата, нощем - обратно.
Скорост: 2-7 m/s (слаб до умерен).
Тип: Периодичен, денонощен цикъл море-суша
Описание: Променя посоката си през денонощието.

=== МУСОН ===
Посока: Летният - от океана към сушата, зимният - обратно.
Скорост: 5-20 m/s, понякога до 30 m/s.
Тип: Периодичен вятър, свързан със сезонни дъждове в Азия
Описание: Променят посоката си сезонно, характерни за Южна и Югоизточна Азия.

=== ФЬОН ===
Посока: От планинските склонове надолу.
Скорост: 10-25 m/s, понякога над 30 m/s.
Тип: Локален, топъл сух вятър
Описание: Топъл, сух вятър от планините към долините.

=== МИСТРАЛ ===
Посока: От север и северозапад към Средиземно море.
Скорост: 15-25 m/s, понякога над 30 m/s.
Тип: Локален, студен планински вятър
Описание: Силен, студен вятър от Алпите към Средиземноморието.

===== ИЗВОД: =====
Изредени са 5 известни вятъра, подредени по скорост:
3 морски (зефир, бриз, мусон) и 2 планински.
Най-лекият и приятен е морският зефир, който присъства в поезията,
а най-силен е алпийският мистрал, който на нася материални щети.

D:\Exercises\11 class\Inheritance\Dynamic_polymorphism\bin\Debug\net8.0\Winds_interfaces.
code 0.
```

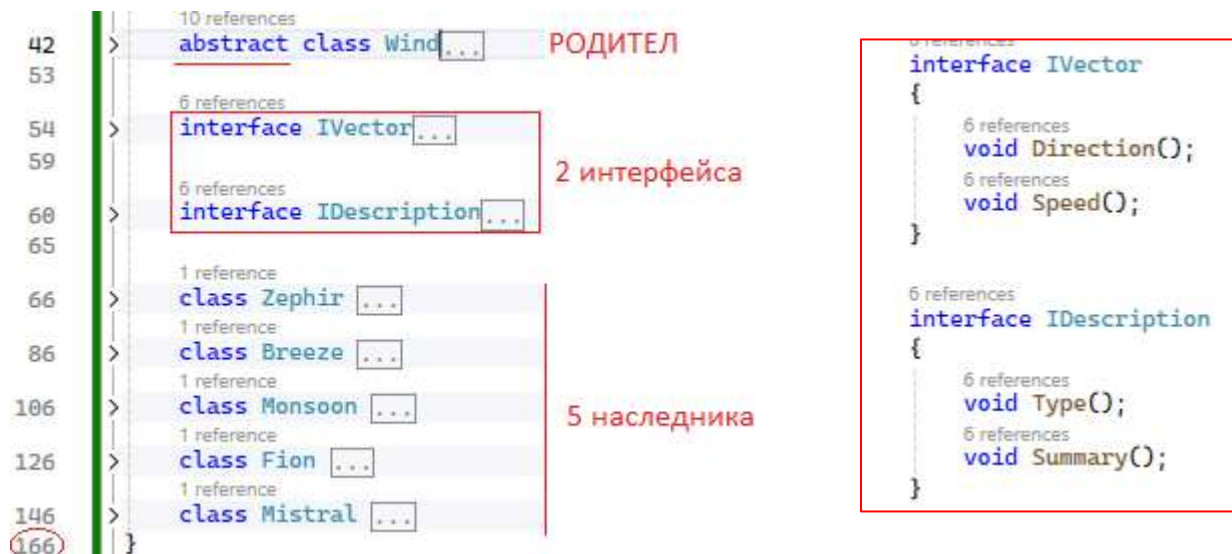
Когато имаме 1 родителски клас, той може да има много класове, които го наследяват (наследници). Дори може да използваме наследници на наследниците (да влагаме класове). При абстрактните родителски класове, можем да използваме абстрактни методи (без тяло). Така запазваме само името на метода, но понеже той няма тяло, наследниците задължително трябва да му го дадат (имплементират).

Една от концепциите на ООП е DRY – Don't Repeat Yourself – да няма повтарящ се код.

Можем да създадем пакет от свойства и имена на методи, който се нарича интерфейс (inter-face –

между лица, междинно онагледяване/визуализиране). Можем да създадем и много пакети от методи и свойства. Хубавото е, че накрая, 1 клас може да наследява само 1 родителски клас, но може да наследява много интерфейси (отделени пакети), които да бъдат **изредени със запетайки**. Когато бъде извикан 1 интерфейс, понеже той също резервира само името на метод, входните променливи и типа на връщаната му стойност, **ЗАДЪЛЖИТЕЛНО** трябва да му се зададе тяло, т.е. в момента, в който решим да извикаме (наследим) даден интерфейс, знаем, че сме длъжни да имплементираме всички методи в него. **Интерфейсите се задават с ключова дума interface и имената им за по-голяма яснота започват с I (interface)**. Един интерфейс може да бъде използван от много класове.

Ще демонстрираме употребата на пакети (интерфейси) чрез програма, която ще даде информация за 5 от най-известните ветрове. Имаме 1 родителски абстрактен клас Wind и 2



Нека родителският клас е абстрактен, съдържа общо заглавие и общ извод, а името на вятъра да бъде задължително и read-only. Единствената особеност е, че заглавието се връща като текст. Това ще ни позволи по-гъвкава обработка в Main(). Нека създадем 2 различни пакета интерфейси, които в случая съдържат само по 2 метода, които отпечатват.

```

abstract class Wind
{
    static - общи за класа
    public static string Heading() => "5-те най-известни вятъра";
    public abstract string Name { get; }
    public static void Conclusion() => Console.WriteLine(@"
===== ИЗВОД: =====
Изредени са 5 известни вятъра, подредени по скорост:
3 морски (зефир, бриз, мусон) и 2 планински.
Най-лекият и приятен е морският зефир, който присъства в поезията,
а най-силен е алпийският мистрал, който на нася материални щети.");
}
  
```

Нека създадем и 5 наследни класа, като обърнем внимание, че докато не се имплементират всички методи, които са заявени като ЗАДЪЛЖИТЕЛНИ от взетия интерфейс, ще се подчертава зигзагообразна грешка за очакване от класа, по същия начин, както когато се очаква ЗАДЪЛЖИТЕЛНА имплементация и на абстрактни методи.

```
class Zephir : Wind, IVector, IDescription 1 клас + 2 интерфейса
{
    2 references
    public override string Name => "Зефир";
    2 references
    public void Direction()
    {
        Console.WriteLine("Посока: Западен Вятър.");
    }
И 1
    2 references
    public void Speed()
    {
        Console.WriteLine("Скорост: 1-5 m/s (лек вятър).");
    }
    2 references
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та интерфейса
        Console.WriteLine("Тип: Локален, приятен лек вятрец");
И 2
    2 references
    public void Summary() =>
        Console.WriteLine("Описание: Лек западен вятър, характерен за Средиземноморието.");
}
```

За по-голяма нагледност кой метод към кой интерфейс принадлежи, съм използвала знак **=>** за методите на IVector (Type() и Summary()), докато на IDescription (Direction() и Speed()) са с **{}**;

Обърнете внимание, че имаме 2 интерфейса, но не сме длъжни да ги наследим всички – може да извикаме само 1 (този, който ни е нужен). Ако го извикаме, обаче, сме длъжни да имплементираме методите му.

Аналогично попълваме с акуратни данни и останалите 4 известни вятъра:

```
1 reference
class Breeze : Wind, IVector, IDescription
{
    2 references
    public override string Name => "Бриз";

    2 references
    public void Direction()
    {
        Console.WriteLine("Посока: Денем от морето към сушата, нощем – обратно.");
    }

    2 references
    public void Speed()
    {
        Console.WriteLine("Скорост: 2-7 m/s (слаб до умерен).");
    }

    2 references
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та интерфейса
        Console.WriteLine("Тип: Периодичен, денонощен цикъл море-суша");

    2 references
    public void Summary() =>
        Console.WriteLine("Описание: Променя посоката си през денонощието.");
}

1 reference
class Monsoon : Wind, IVector, IDescription
{
    2 references
    public override string Name => "Мусон";

    2 references
    public void Direction()
    {
        Console.WriteLine("Посока: Летният – от океана към сушата, зимният – обратно.");
    }

    2 references
    public void Speed()
    {
        Console.WriteLine("Скорост: 5-20 m/s, понякога до 30 m/s.");
    }

    2 references
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та интерфейса
        Console.WriteLine("Тип: Периодичен вятър, свързан със сезонни дъждове в Азия");

    2 references
    public void Summary() =>
        Console.WriteLine("Описание: Променят посоката си сезонно, характерни за Южна и Югоизточна Азия.");
}
```

```

class Fion : Wind, IVector, IDescription
{
    2 references
    public override string Name => "Фьон";

    2 references
    public void Direction()
    {
        Console.WriteLine("Посока: От планинските склонове надолу.");
    }

    2 references
    public void Speed()
    {
        Console.WriteLine("Скорост: 10-25 m/s, понякога над 30 m/s.");
    }

    2 references
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та интерфейса
        Console.WriteLine("Тип: Локален, топъл сух вятър");

    2 references
    public void Summary() =>
        Console.WriteLine("Описание: Топъл, сух вятър от планините към долините.");
}

class Mistral : Wind, IVector, IDescription
{
    2 references
    public override string Name => "Мистрал";

    2 references
    public void Direction()
    {
        Console.WriteLine("Посока: От север и северозапад към Средиземно море.");
    }

    2 references
    public void Speed()
    {
        Console.WriteLine("Скорост: 15-25 m/s, понякога над 30 m/s.");
    }

    2 references
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та интерфейса
        Console.WriteLine("Тип: Локален, студен планински вятър");

    2 references
    public void Summary() =>
        Console.WriteLine("Описание: Силен, студен вятър от Алпите към Средиземноморието.");
}

```


5-те класа са шаблонни, остава само имплементацията им като обекти в Main():

```
static void Main(string[] args)
{
    List<Wind> winds = new List<Wind>
    {
        new Zephir(),
        new Breeze(),
        new Monsoon(),
        new Fion(),
        new Mistral(),
    };

    Console.WriteLine($"=== {Wind.Heading().ToUpper()} ===");
    Console.WriteLine(new string('=', Wind.Heading().Length + 8));
}
```

3 = + 1 интервал
4 static 4
4 вместо число

Остава да обходим всички елементи и за всеки да изредим методите от интерфейсите му:

```
foreach (var wind in winds)
{
    Console.WriteLine($"\\n=== {wind.Name.ToUpper()} ===");

    if (wind is IVector vector)
    {
        vector.Direction();
        vector.Speed();
    }

    if (wind is IDescription description)
    {
        description.Type();
        description.Summary();
    }

    Wind.Conclusion();
}
```

не са тип РОДИТЕЛ,
а свой собствен тип
(шаблон)
2 интерфейса
static извод

Където is е проверка за булева стойност:

```
if (wind is IVector vector)
{
    vector
    vector
}
```

readonly struct System.Boolean
Represents a Boolean (true or false) value.

Поздрав с естрадна песен от 1970 г – [Паша Христова – „Повея, ветре“](#)

Целият код:

```
using System;
using System.Collections.Generic;

namespace Winds
{
    internal class Program
```

```

{
    static void Main(string[] args)
    {
        List<Wind> winds = new List<Wind>
        {
            new Zephir(),
            new Breeze(),
            new Monsoon(),
            new Fion(),
            new Mistral(),
        };

        Console.WriteLine($"=== {Wind.Heading().ToUpper()} ===");
        Console.WriteLine(new string('=', Wind.Heading().Length + 8));

        foreach (var wind in winds)
        {
            Console.WriteLine($"\\n=== {wind.Name.ToUpper()} ===");

            if (wind is IVector vector)
            {
                vector.Direction();
                vector.Speed();
            }

            if (wind is IDescription description)
            {
                description.Type();
                description.Summary();
            }

            Wind.Conclusion();
        }
    }
}

abstract class Wind
{
    public static string Heading() => "5-те най-известни вятъра";
    public abstract string Name { get; }
    public static void Conclusion() => Console.WriteLine(@"
===== ИЗВОД: =====
Изредени са 5 известни вятъра, подредени по скорост:
3 морски (зефир, бриз, мусон) и 2 планински.
Най-лекият и приятен е морският зефир, който присъства в поезията,
а най-силен е алпийският мистрал, който на нася материални щети.");
}

interface IVector
{
    void Direction();
    void Speed();
}

interface IDescription
{
    void Type();
    void Summary();
}

```

```

class Zephir : Wind, IVector, IDescription
{
    public override string Name => "Зефир";
    public void Direction()
    {
        Console.WriteLine("Посока: Западен вятър.");
    }

    public void Speed()
    {
        Console.WriteLine("Скорост: 1-5 м/с (лек вятър).");
    }
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та
интерфейса Console.WriteLine("Тип: Локален, приятен лек ветрец");

    public void Summary() =>
        Console.WriteLine("Описание: Лек западен вятър, характерен за
Средиземноморието.");
}
class Breeze : Wind, IVector, IDescription
{
    public override string Name => "Бриз";

    public void Direction()
    {
        Console.WriteLine("Посока: Денем от морето към сушата, нощем -
обратно.");
    }

    public void Speed()
    {
        Console.WriteLine("Скорост: 2-7 м/с (слаб до умерен).");
    }
    public void Type() => // еквивалентен запис на {}, за разлика между 2-та
интерфейса Console.WriteLine("Тип: Периодичен, денонощен цикъл море-суша");

    public void Summary() =>
        Console.WriteLine("Описание: Променя посоката си през денонощието.");
}
class Monsoon : Wind, IVector, IDescription
{
    public override string Name => "Мусон";

    public void Direction()
    {
        Console.WriteLine("Посока: Летният - от океана към сушата, зимният -
обратно.");
    }

    public void Speed()
    {
        Console.WriteLine("Скорост: 5-20 м/с, понякога до 30 м/с.");
    }
}

```



```

        public void Type() => // еквивалентен запис на {}, за разлика между 2-та
интерфейса
        Console.WriteLine("Тип: Периодичен вятър, свързан със сезонни дъждове в Азия");

        public void Summary() =>
        Console.WriteLine("Описание: Променят посоката си сезонно, характерни за
Южна и Югоизточна Азия.");
    }
    class Fion : Wind, IVector, IDescription
    {
        public override string Name => "Фьон";

        public void Direction()
        {
            Console.WriteLine("Посока: От планинските склонове надолу.");
        }

        public void Speed()
        {
            Console.WriteLine("Скорост: 10-25 m/s, понякога над 30 m/s.");
        }

        public void Type() => // еквивалентен запис на {}, за разлика между 2-та
интерфейса
        Console.WriteLine("Тип: Локален, топъл сух вятър");

        public void Summary() =>
        Console.WriteLine("Описание: Топъл, сух вятър от планините към
долините.");
    }
    class Mistral : Wind, IVector, IDescription
    {
        public override string Name => "Мистрал";

        public void Direction()
        {
            Console.WriteLine("Посока: От север и северозапад към Средиземно
море.");
        }

        public void Speed()
        {
            Console.WriteLine("Скорост: 15-25 m/s, понякога над 30 m/s.");
        }

        public void Type() => // еквивалентен запис на {}, за разлика между 2-та
интерфейса
        Console.WriteLine("Тип: Локален, студен планински вятър");

        public void Summary() =>
        Console.WriteLine("Описание: Силен, студен вятър от Алпите към
Средиземноморието.");
    }
}

```